Spring Cloud Netflix Eureka

主要内容

Eureka 简介
Eureka 和 Zookeeper 对比
搭建 Eureka 注册中心
Eureka 服务管理平台介绍
搭建高可用集群
集群原理
Eureka 优雅停服

学习目标

知识点	要求
Eureka 简介	了解
Eureka 和 Zookeeper 对比	掌握
搭建 Eureka 注册中心	掌握
Eureka 服务管理平台介绍	了解
搭建高可用集群	掌握
集群原理	掌握
Eureka 优雅停服	掌握

Spring Cloud Netflix Eureka

一、 Eureka 简介

Eureka 是 Netflix 开发的服务发现*框架*，本身是一个基于 REST 的*服务*，主要用于定位运行在 AWS 域中的中间层服务，以达到负载均衡和中间层服务故障转移的目的。SpringCloud 将它集成在其子项目 spring-cloud-netflix 中，以实现 SpringCloud 的服务发现功能。

1 Eureka 组件

Eureka 包含两个组件：Eureka Server 和 Eureka Client。

1.1 Eureka Server

Eureka Server 提供服务注册服务，各个节点启动后，会在 Eureka Server 中进行注册，这样 EurekaServer 中的服务注册表中将会存储所有可用服务节点的信息，服务节点的信息

可以在界面中直观的看到。

Eureka Server 本身也是一个服务，默认情况下会自动注册到 Eureka 注册中心。

如果搭建单机版的 Eureka Server 注册中心，则需要配置取消 Eureka Server 的自动注册逻辑。毕竟当前服务注册到当前服务代表的注册中心中是一个说不通的逻辑。

Eureka Server 通过 Register、Get、Renew 等接口提供服务的注册、发现和心跳检测等服务。

1.2 Eureka Client

Eureka Client 是一个 java 客户端，用于简化与 Eureka Server 的交互，客户端同时也具备一个内置的、使用轮询(round-robin)负载算法的负载均衡器。在应用启动后，将会向 Eureka Server 发送心跳,默认周期为 30 秒，如果 Eureka Server 在多个心跳周期内没有接收到某个节点的心跳，Eureka Server 将会从服务注册表中把这个服务节点移除(默认 90 秒)。

Eureka Client 分为两个角色，分别是：Application Service(Service Provider) 和 Application Client(Service Consumer)

1.2.1 Application Service

服务提供方，是注册到 Eureka Server 中的服务。

1.2.2 Application Client

服务消费方，通过 Eureka Server 发现服务，并消费。

注意：在这里，Application Service 和 Application Client 不是绝对上的定义，因为 Provider 在提供服务的同时，也可以消费其他 Provider 提供的服务；Consumer 在消费服务的同时，也可以提供对外服务。

二、 Eureka 和 Zookeeper 对比

1 什么是 CAP 定理

CAP 原则又称 CAP 定理，指的是在一个分布式系统中，Consistency（数据一致性）、Availability（服务可用性）、Partition tolerance（分区容错性），三者不可兼得。

CAP 由 Eric Brewer 在 2000 年 PODC 会议上提出。该猜想在提出两年后被证明成立，成为我们熟知的 CAP 定理

分布式系统 CAP 定理	
C 数据一致性 (Consistency)	也叫做数据原子性 系统在执行某项操作后仍然处于一致的状态。在分布式系统中，更新操作执行成功后所有的用户都应该读到最新的值，这样的系统被认为是具有强一致性的。等同于所有节点访问同一份最新的数据副本。 优点： 数据一致，没有数据错误可能。 缺点： 相对效率降低。

A 服务可用性 (Availablity)	每一个操作总是能够在一定的时间内返回结果，这里需要注意的是"一定时间内"和"返回结果"。一定时间内指的是，在可以容忍的范围内返回结果，结果可以是成功或者是失败。
P 分区容错性 (Partition-torlerance)	在网络分区的情况下，被分隔的节点仍能正常对外提供服务(分布式集群，数据被分布存储在不同的服务器上，无论什么情况，服务器都能正常被访问)

定律：任何分布式系统只可同时满足二点，***没法三者兼顾***。	
CA，放弃 P	如果想避免分区容错性问题的发生，一种做法是将所有的数据(与事务相关的)/服务都放在一台机器上。虽然无法100%保证系统不会出错，但不会碰到由分区带来的负面效果。当然这个选择会严重的影响系统的扩展性。
CP，放弃 A	相对于放弃"分区容错性"来说，其反面就是放弃可用性。一旦遇到分区容错故障，那么受到影响的服务需要等待一定时间，因此在等待时间内系统无法对外提供服务。
AP，放弃 C	这里所说的放弃一致性，并不是完全放弃数据一致性，而是放弃数据的强一致性，而保留数据的最终一致性。以网络购物为例，对只剩下一件库存的商品，如果同时接受了两个订单，那么较晚的订单将被告知商品告罄。

2 基于 CAP 定理比对 Eureka 和 Zookeeper

对比项	Zookeeper	Eureka	描述
CAP	CP	AP	ZooKeeper 分布集群是使用主从模型实现的。在一个时间点上，只有一个 leader 真正的对外提供服务。其他的 follower 都会实时备份 leader 中的数据，当 leader 宕机，则 follower 选举出新的 leader 对外提供服务。 Eureka 分布集群是平等模型（无主模型），所有的节点都是平等的，客户端访问任意节点都可以提供实时的服务响应。如果某节点发送宕机等故障，接收到的请求会转交给其他的节点。无主模型，每个节点的数据可能不实时

			一致，节点需要通过网络通讯从其他节点获取数据，并实现数据的一致。可能有网络延迟或网络故障或通讯频率问题。
Dubbo 集成	已支持	-	Dubbo 开发的时候不需要考虑注册中心选择。 Spring Cloud 中推荐使用 Eureka 作为注册中心，Eureka 是由 Spring Cloud 子项目 spring-cloud-netflix 集成的。是 Spring Cloud 中的一个组件，会有针对性的服务提供和发现组件。
Spring Cloud 集成	已支持	已支持	
kv 服务	支持	-	ZK 支持数据存储，eureka 不支持
使用接口 (多语言能力)	提供客户端	http 多语言	ZK 的跨语言支持比较弱
watch 支持	支持	支持	什么是 Watch 支持？就是客户端监听服务端的变化情况。 zk 通过订阅监听来实现 eureka 通过轮询的方式来实现
集群监控	_	metrics	metrics，运维者可以收集并报警这些度量信息达到监控目的

三、 搭建 Eureka 注册中心

Eureka Server 既是一个注册中心，同时也是一个服务。那么搭建 Eureka Server 的方式和以往搭建 Dubbo 注册中心 ZooKeeper 的方式必然不同，那么首先搭建一个单机版的 Eureka Server 注册中心。

1 POM 文件

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.2.2.RELEASE</version>
    </parent>
    <groupId>com.bjsxt</groupId>
    <artifactId>cloudeureka</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
        </dependency>
    </dependencies>
</project>
```

2 配置文件 application.yml

Eureka Server 本身也是一个服务，同时又是一个注册中心。在 Spring Cloud 中，启动的微服务会自动的搜索注册中心并注册服务，那么在单机版 Eureka Server 环境中，当前服务注册到当前服务中，明显是不合适的。所以搭建 Eureka Server 单机版时，需要提供特殊的全局配置，避免回路注册逻辑。

同理，Eureka Server 服务在注册中心中发现服务列表逻辑也是不必要的。毕竟注册中心是一个中立的服务管理平台，如果是单机版 Eureka Server 环境中，Eureka Server 服务再去发现服务列表，明显也是不必要的。也需要通过全局配置，避免回路发现逻辑。

```
server:
  # 设置 Eureka Server WEB 控制台端口、服务注册发现端口。Eureka Server 服务注册
```

```
发现端口默认为8761。
  port: 8761
spring:
  application:
    # 设置spring应用命名，默认为null。同命名的应用会注册到同一个服务集群中。
    name: cloud-eureka
eureka:
  client:
    # 是否将自己注册到Eureka-Server中，默认的为true
    register-with-eureka: false
    # 是否从Eureka-Server中获取服务注册信息，默认为true
    fetch-registry: false
```

如果需要修改 Eureka Server 服务注册发现端口，具体配置如下：

```
server:
  # 设置Eureka Server WEB控制台端口、服务注册发现端口。Eureka Server服务注册发现端口默认为8761。
  port: 8080
spring:
  application:
    # 设置spring应用命名，默认为null。同命名的应用会注册到同一个服务集群中。
    name: cloud-eureka
eureka:
  client:
    # 是否将自己注册到Eureka-Server中，默认的为true
    register-with-eureka: false
    # 是否从Eureka-Server中获取服务注册信息，默认为true
    fetch-registry: false
    service-url:
      defaultZone: http://localhost:8080/eureka/
```

3 启动类

```
package com.bjsxt.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

/**
 * @EnableEurekaServer - 启动时初始化Eureka Server注册中心。
 */
@SpringBootApplication
```

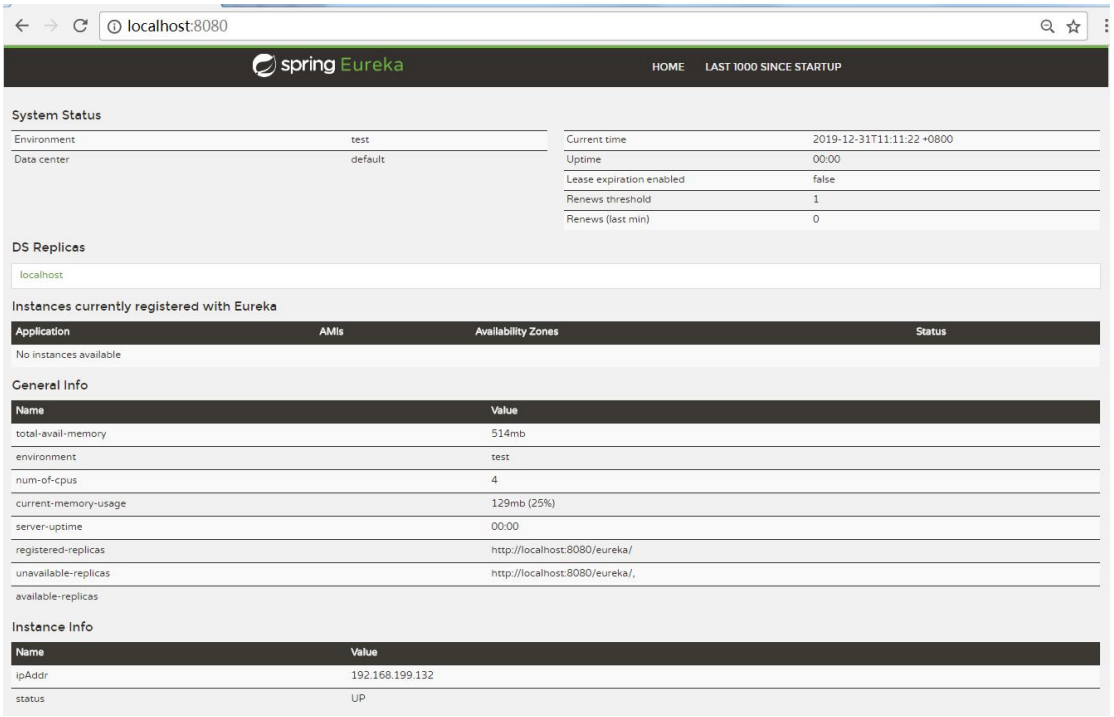
```
@EnableEurekaServer
public class EurekaApp {
    public static void main(String[] args) {
        SpringApplication.run(EurekaApp.class, args);
    }
}
```

4 访问 Eureka Server WEB 服务管理平台

访问服务管理平台地址为： http://ip:port/

四、 Eureka 服务管理平台介绍

1 Eureka Server 服务管理平台访问预览



2 System Status

系统状态展示

3 DS Replicas

注册中心集群列表

4 Instances currently registered with Eureka

已在注册中心中注册的服务列表

5 General Info

当前注册中心相关信息展示

6 Instance Info

当前注册中心实例信息展示

五、 搭建高可用集群

1 在 Eureka 应用中定义多环境配置

1.1 application-eureka1.yml

```yaml
server:
  # 设置 Eureka Server WEB 控制台端口、服务注册发现端口。Eureka Server 服务注册发现端口默认为 8761。
  port: 8761
spring:
  application:
    # 设置 spring 应用命名，默认为 null。同命名的应用会注册到同一个服务集群中。
    name: cloud-eureka
eureka:
  client:
    service-url:
      defaultZone:
        - http://eureka2:8761/eureka/
  instance:
    hostname: eureka1
```

1.2 application-eureka2.yml

```yaml
server:
  # 设置 Eureka Server WEB 控制台端口、服务注册发现端口。Eureka Server 服务注册发现端口默认为 8761。
  port: 8761
spring:
  application:
```

```yaml
    # 设置 spring 应用命名，默认为 null。同命名的应用会注册到同一个服务集群中。
    name: cloud-eureka
eureka:
  client:
    service-url:
      defaultZone:
        - http://eureka1:8761/eureka/
  instance:
    hostname: eureka2
```

2 打包工程

2.1 POM 依赖

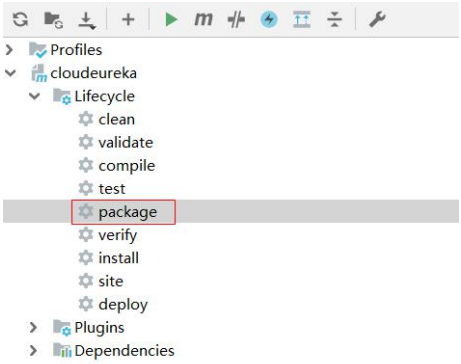
增加 Spring Boot 打包插件

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.2.2.RELEASE</version>
    </parent>
    <groupId>com.bjsxt</groupId>
    <artifactId>cloudeureka</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
    <dependencies>
        <dependency>
```

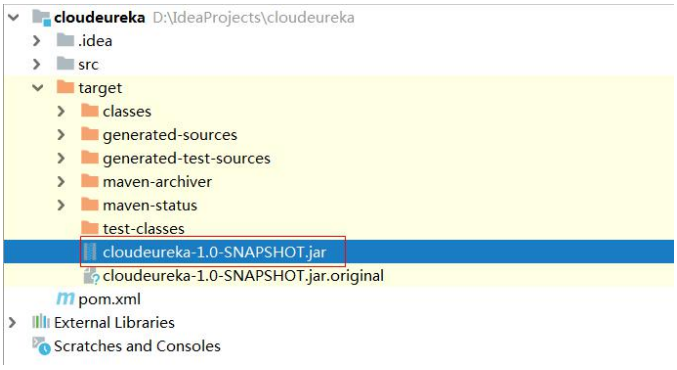
```
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

2.2 打包



2.3 打包结果



3 上传打包后的 jar 文件到 Linux 系统

4 设置 Linux 主机域名

修改/etc/hosts 文件，设置主机域名。将主机域名和 IP 进行绑定。

vi /etc/hosts

新增内容如下：两个 Linux 系统修改内容相同。（***IP 根据具体环境配置***）

```
192.168.17.10 eureka1
192.168.17.20 eureka2
```

5 启动 Eureka

5.1 使用 java 命令启动

java -jar -Dspring.profiles.active=eureka1 cloudeureka-1.0-SNAPSHOT.jar

5.2 脚本启动

当然，每次通过命令来启动 Eureka Server 过于麻烦，可以定义一个 shell 文件来简化操作。（Java 程序员不必要掌握 shell 脚本的编写。建议对 shell 脚本有一定的了解，至少能够修改内部的必要参数值。Shell 脚本一般由运维或部署人员开发。也可能有系统工程师开发。）具体 shell 内容如下：

```
#!/bin/bash

cd `dirname $0`

CUR_SHELL_DIR=`pwd`
CUR_SHELL_NAME=`basename ${BASH_SOURCE}`

JAR_NAME="项目 jar 包名称"
JAR_PATH=$CUR_SHELL_DIR/$JAR_NAME

#JAVA_MEM_OPTS=" -server -Xms1024m -Xmx1024m -XX:PermSize=128m"
JAVA_MEM_OPTS=""

SPRING_PROFILES_ACTIV="-Dspring.profiles.active=配置文件变量名称"
#SPRING_PROFILES_ACTIV=""
LOG_DIR=$CUR_SHELL_DIR/logs
LOG_PATH=$LOG_DIR/eureka-server.log

echo_help()
{
    echo -e "syntax: sh $CUR_SHELL_NAME start|stop"
}
```

```
    if [ -z $1 ];then
        echo_help
        exit 1
    fi

    if [ ! -d "$LOG_DIR" ];then
        mkdir "$LOG_DIR"
    fi

    if [ ! -f "$LOG_PATH" ];then
        touch "$LOG_DIR"
    fi

    if [ "$1" == "start" ];then

        # check server
        PIDS=`ps --no-heading -C java -f --width 1000 | grep $JAR_NAME | awk '{print $2}'`
        if [ -n "$PIDS" ]; then
            echo -e "ERROR: The $JAR_NAME already started and the PID is ${PIDS}."
            exit 1
        fi

        echo "Starting the $JAR_NAME..."

        # start
        nohup java $JAVA_MEM_OPTS -jar $SPRING_PROFILES_ACTIV $JAR_PATH >>
$LOG_PATH 2>&1 &

        COUNT=0
        while [ $COUNT -lt 1 ]; do
            sleep 1
            COUNT=`ps    --no-heading -C java -f --width 1000 | grep "$JAR_NAME" | awk
'{print $2}' | wc -l`
            if [ $COUNT -gt 0 ]; then
                break
            fi
        done
        PIDS=`ps    --no-heading -C java -f --width 1000 | grep "$JAR_NAME" | awk '{print
$2}'`
        echo "${JAR_NAME} Started and the PID is ${PIDS}."
        echo "You can check the log file in ${LOG_PATH} for details."

    elif [ "$1" == "stop" ];then
```

```
        PIDS=`ps --no-heading -C java -f --width 1000 | grep $JAR_NAME | awk '{print $2}'`
        if [ -z "$PIDS" ]; then
            echo "ERROR:The $JAR_NAME does not started!"
            exit 1
        fi

        echo -e "Stopping the $JAR_NAME..."

        for PID in $PIDS; do
            kill $PID > /dev/null 2>&1
        done

        COUNT=0
        while [ $COUNT -lt 1 ]; do
            sleep 1
            COUNT=1
            for PID in $PIDS ; do
                PID_EXIST=`ps --no-heading -p $PID`
                if [ -n "$PID_EXIST" ]; then
                    COUNT=0
                    break
                fi
            done
        done

        echo -e "${JAR_NAME} Stopped and the PID is ${PIDS}."
    else
        echo_help
        exit 1
    fi
```

设置好 shell 启动脚本后，需要提供可执行权限：shell 脚本文件名自己修改。

chmod 755 xxx.sh

脚本使用方式为：

启动 Eureka Server 命令：　./xxx.sh start

关闭 Eureka Server 命令：　./xxx.sh stop

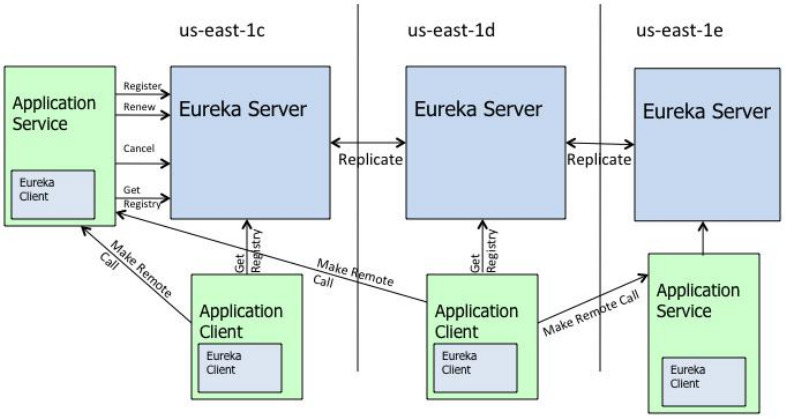
六、 集群原理

Eureka Server 注册中心的集群和 Dubbo 的 ZooKeeper 注册中心集群在结构上有很大的不同。

Eureka Server 注册中心集群中每个节点都是平等的，集群中的所有节点同时对外提供服务的发现和注册等功能。同时集群中每个 Eureka Server 节点又是一个微服务，也就是说，

每个节点都可以在集群中的其他节点上注册当前服务。又因为每个节点都是注册中心，所以节点之间又可以相互注册当前节点中已注册的服务，并发现其他节点中已注册的服务。

1 Eureka 集群架构原理图



七、 Eureka 优雅停服

1 自我保护模式

一般情况下，微服务在 Eureka 上注册后，会每 30 秒发送心跳包，Eureka 通过心跳来判断服务时候健康，同时会定期删除超过 90 秒没有发送心跳服务。

有两种情况会导致 Eureka Server 收不到微服务的心跳：一是微服务自身的原因；二是微服务与 Eureka 之间的网络故障。

通常微服务自身的故障关闭只会导致个别服务出现故障，一般不会出现大面积故障，而网络故障通常会导致 Eureka Server 在短时间内无法收到大批心跳。考虑到这个区别，Eureka 设置了一个阀值，当判断离线服务的数量超过阀值时，Eureka Server 认为很大程度上出现了网络故障，将不再删除心跳过期的服务。

那么这个阀值是多少呢？15 分钟之内是否低于 85%；Eureka Server 在运行期间，会统计心跳失败的比例在 15 分钟内是否低于 85%，这种算法叫做 Eureka Server 的自我保护模式。

2 为什么要自我保护

因为同时保留"好数据"与"坏数据"总比丢掉任何数据要更好，当网络故障恢复后，这个 Eureka 节点会退出"自我保护模式"。Eureka 还有客户端缓存功能(也就是微服务的缓存功能)。即便 Eureka 集群中所有节点都宕机失效，微服务的 Provider 和 Consumer 都能依托服务缓存正常通信。微服务的负载均衡策略会自动剔除离线的微服务节点。

3 关闭自我保护

修改 Eureka Server 中的核心配置文件 application.yml

```yaml
server:
  # 设置 Eureka Server WEB 控制台端口、服务注册发现端口。Eureka Server 服务注册发现端口默认为 8761。
  port: 8761
spring:
  application:
    # 设置 spring 应用命名，默认为 null。同命名的应用会注册到同一个服务集群中。
    name: cloud-eureka
eureka:
  client:
    service-url:
      defaultZone:
        - http://eureka2:8761/eureka/
  instance:
    hostname: eureka1
  server:
    # 是否开启自我保护， 默认为 true
    enable-self-preservation: false
    # 清理间隔，单位是毫秒。默认值为 1 分钟
    eviction-interval-timer-in-ms: 60000
```